



Faculty of Engineering
Computer Engineering Department

Phase 2

Satellite Imagery Project

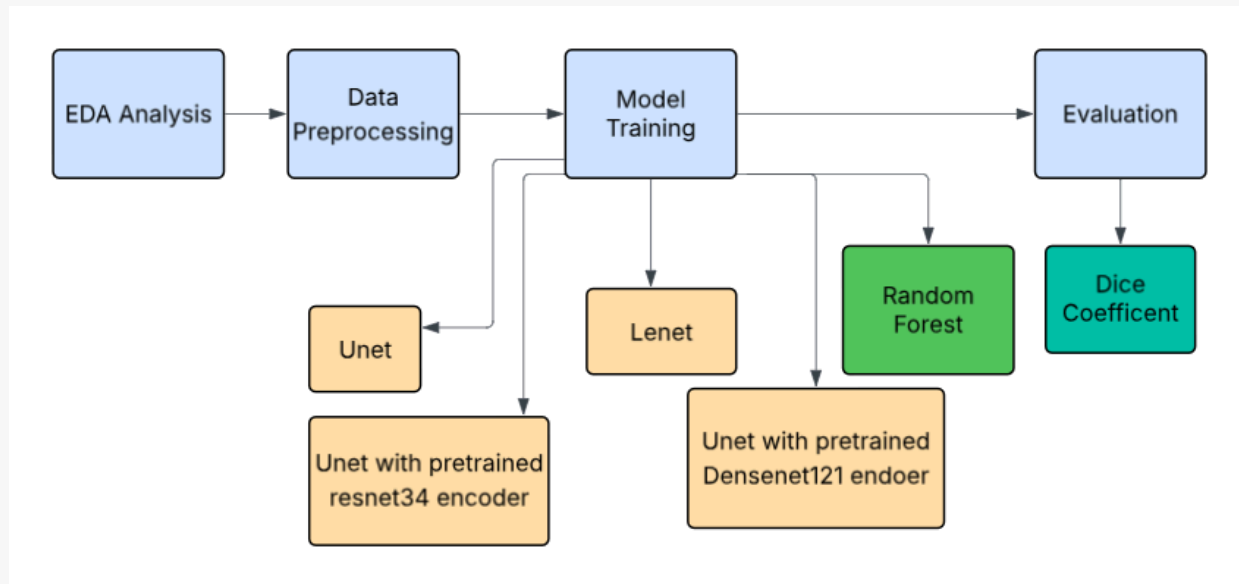
Cloud Segmentation

Team 12

Presented by

Name	Section	BN
Mohamed Yehia Alahmad	2	14
Moustafa Mohammed Elsayed	2	23
Mostafa Hani Mostafa	2	24
Mennatallah Ahmed Moustafa	2	25

I. Project Pipeline



II. Literature Review

Introduction

Before starting our project, we studied existing research to understand how others solved cloud detection. We looked at classical methods like Fmask and Random Forest, as well as deep learning models like LeNet, UNet, and DeepLab v3+. This review summarizes what we found and why we chose to try Random Forest, LeNet, UNet with ResNet and DenseNet121 backbones, and DeepLab v3+ with ResNet and EfficientNet-B7 backbones.

Classical Methods

We first explored classical methods that use simple rules to detect clouds. One popular method is Fmask, which uses spectral tests to find clouds in Landsat images ([Zhu and Woodcock, 2012, Object-based cloud and cloud shadow detection in Landsat imagery - ScienceDirect](#)). Fmask works well, the average Fmask overall cloud accuracy is as high as 96.4%, but it often confuses bright surfaces like snow or buildings with clouds. This limitation pushed us to try another machine-learning approach.

We also studied Random Forest, a classical machine-learning method that classifies pixels based on spectral features. Although it doesn't capture spatial patterns well, Random Forest is easy to use and can handle some variability in data. So we decided to use it as our classical method.

Deep Learning Methods

Since classical methods had limitations, we looked at deep learning models, which are better at handling complex patterns in images. We focused on LeNet, UNet, and DeepLab v3+, as they are commonly used for tasks like cloud detection.

1. LeNet

LeNet is a simple convolutional neural network (CNN) designed for basic image tasks. We considered it as a starting point because it's lightweight. However, a review by Mahajan and Fataniya (2019, [Cloud detection methodologies: variants and development—a review | Complex & Intelligent Systems](#)) noted that LeNet struggles with complex segmentation tasks like cloud detection due to its shallow structure. This made us expect limited performance but still worth trying as a baseline.

1. UNet-based Models

UNet is a popular model for image segmentation, with an encoder-decoder design that captures both details and context. A study by Zhang et al. (2020, [Lightweight U-Net for cloud detection of visible and thermal infrared remote sensing images | Optical and Quantum Electronics](#)) used UNet with optimized backbones, which inspired us to try UNet with ResNet and DenseNet121. ResNet's deep layers help with feature extraction, while DenseNet121's dense connections improve efficiency, making them good fits for our project.

3. DeepLab v3+ Models

DeepLab v3+ is another strong model for segmentation, using atrous spatial pyramid pooling to capture multi-scale features. Li et al. (2021, [Light-Weight Cloud Detection Network for Optical Remote Sensing Images with Attention-Based DeeplabV3+ Architecture](#)) developed CD-AttDLV3+, a DeepLab v3+ model with a MobileNetV2 backbone, achieving an F1 score of 0.9499 on a Landsat dataset. This showed DeepLab v3+ could work well for our task. We also found a 2023 study ([Cloud Detection Method Based on Improved DeeplabV3+ Remote Sensing Image | IEEE Journals & Magazine | IEEE Xplore](#)) that improved DeepLab v3+ with an Xception backbone. These findings encouraged us to try DeepLab v3+ with ResNet and EfficientNet-B7 backbones, though we noted that EfficientNet-B7 might be heavy for our hardware.

Conclusion

Our review showed that classical methods like Fmask and Random Forest are simple but not great for complex scenes with thin clouds or bright surfaces. Deep learning models like UNet and DeepLab v3+ perform better because they use both spectral and spatial information. LeNet was included as a simple baseline, but we expected UNet and DeepLab v3+ to outperform it. The success of ResNet, DenseNet121, and EfficientNet-B7 backbones in prior studies made them logical choices for our dataset, which has similar band configurations.

This guided us to try Random Forest as a classical method and LeNet, UNet, and DeepLab v3+ as deep learning models. UNet and DeepLab v3+ stood out as the best options for cloud detection, especially with backbones like ResNet, DenseNet121, and EfficientNet-B7. These models are well-suited for our dataset and project goals, giving us confidence in our approach.

III. Data Exploration

We conducted an exploratory data analysis (EDA) on the dataset. Below is a summary of the steps:

1. Image Count and Mask Verification

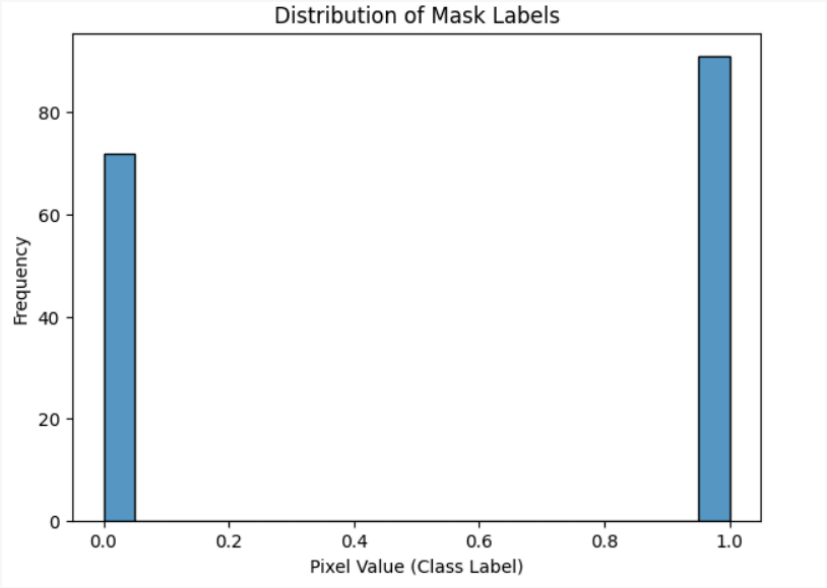
We counted the total number of images in the dataset and checked for images missing masks. No images were found without a corresponding mask.

2. Duplicate Image Check

We examined the dataset for duplicate images (identical image and mask pairs) using hashing of images. Only two instances of duplicated images and masks were identified.

3. Mask Pixel Value Distribution

A histogram was plotted to show the distribution of pixel values in the masks.



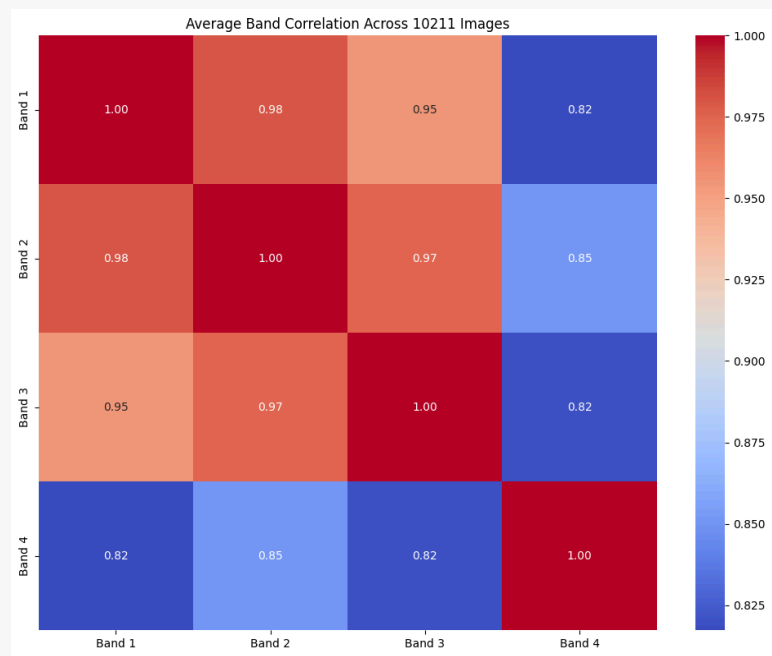
4. Cloud Coverage Distribution

We analyzed the distribution of images based on cloud coverage and categorized them into three groups:

Total images (after filtering mislabeled)	10211
Fully Cloudy	3528
Partially Cloudy	5888
Cloud-Free	795

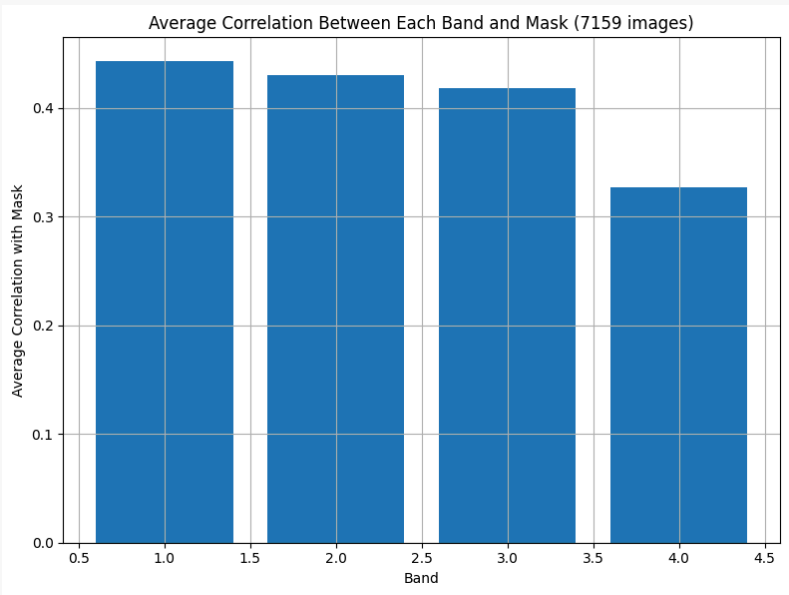
5. Band Correlation Analysis

We calculated the average correlation between image bands. Bands 1 and 4 were found to be the least correlated.



6. Bands and Masks correlation

We calculated the average correlation between each image band and the mask and found channels 1, 2, and 3 to be more correlated with the mask than channel 4



7. Random Image Visualization

We selected a random sample of images and visualized them in RGB. This revealed several instances of **mislabeled data** in the dataset.

IV. Preprocessing

1. Data Augmentation Evaluation

We considered applying data augmentation to address potential model misclassifications. However, analysis showed that misclassifications were not due to data imbalance, so we decided not to use data augmentation.

2. Band Selection Based on Correlation

We analyzed the correlation between image bands and found that bands 1, 2, and 3 were highly correlated. To reduce redundancy, we selected only bands 1 and 4 for further processing.

3. Mislabeled Image Filtering

We manually reviewed and filtered out mislabeled images, focusing on those we were confident were incorrectly labeled to improve dataset quality. Also later found the model being confused by other mislabeled images thus we manually filtered only part of the images which are misclassified in training.

4. Image Resizing for Computational Efficiency

For computationally intensive models like DeepLabV3, we encountered out-of-memory errors on the GPU. To address this, we resized images to 70% of their original size, reducing memory usage. We also ensured that images and masks were moved to the GPU for each batch during training.

V. Model Selection and Training

Our model is based on U-Net, a symmetric encoder-decoder architecture suitable for pixel-level prediction tasks, with DenseNet-121 as the encoder.

Why DenseNet-121?

DenseNet-121 (Densely Connected Convolutional Networks) has these advantages:

- Dense connections between layers: Each layer receives inputs from all previous layers.
- Efficient feature reuse leads to better parameter efficiency and gradient flow.
- Pretrained on ImageNet (usually), which helps the model learn better representations quickly (transfer learning).

As previously mentioned, we limited our input to channels 1 and 4, as they demonstrated the lowest correlation and provided the most independent information. We also used the full image resolution of 512×512 during training. To further refine predictions, a confidence threshold of 0.7 was applied to suppress uncertain outputs.

Using a low learning rate of 0.00025 and training for 30 epochs, we achieved the following results:

- Confidence Threshold = 0.7
 - Training Set: Accuracy = 94.00%, **Dice Score = 0.8994**
 - Validation Set: Accuracy = 94.14%, **Dice Score = 0.9024**
 - Test Set: Accuracy = 93.44%, **Dice Score = 0.8937**

This model configuration achieved a public score of **0.79** and a private score of **0.77**, placing **8th on the leaderboard**. The variation between our internal test results and the leaderboard scores can be attributed to **distributional differences** between our test set and the evaluation set used on the leaderboard.

Concerning the Classical approach:

- We used a Random Forest model but trained, validated, and tested it on a subset of the dataset due to constraints in memory and available time. To reduce resource consumption, we limited the hyperparameter search space as follows:
 - `n_estimators`: [30, 50]
 - `max_depth`: [5, 8].
 - `max_samples`: [0.3]
- Results:
 - Params {'n_estimators': 50, 'max_depth': 8, 'max_samples': 0.3} -> Validation Dice: 0.6350
 - Params {'n_estimators': 50, 'max_depth': 5, 'max_samples': 0.3} -> Validation Dice: 0.6341
 - Params {'n_estimators': 30, 'max_depth': 8, 'max_samples': 0.3} -> Validation Dice: 0.6348
 - Params {'n_estimators': 30, 'max_depth': 5, 'max_samples': 0.3} -> Validation Dice: 0.6341

VI. Model Evaluation (Performance Analysis)

We conducted a series of experiments to evaluate model performance, testing different architectures, hyperparameters, and preprocessing strategies. Below is a summary of the trials, their results (measured primarily using the Dice coefficient), and a justification for each move.

Trial 1: Baseline LeNet Model

Justification: LeNet was chosen as a simple starting point to establish a baseline performance with minimal complexity.

- Used the simplest LeNet model with only channel 1, 20 epochs, CrossEntropyLoss, and a learning rate of 0.001 (best hyperparameter).

Results

- ◆ Train Dice: **0.8426**
- ◆ Validation Dice: **0.8284**
- ◆ Test Dice: **0.8377**

Trial 2: Adding Channel 4

Justification: Adding channel 4 taking advantage of low-correlation bands to capture more relevant features, improving performance.

- Improved Trial 1 by using channels 1 and 4, testing multiple hyperparameters.

Results

- ◆ Train Dice: **0.8447**
- ◆ Validation Dice: **0.8400**
- ◆ Test Dice: **0.8497**

Observation: Slight improvement in validation and test scores.

Trial 3: Removing Misabeled Data

Justification: Filtering mislabeled data was tested to reduce noise and improve model generalization.

- Removed ~200 confidently mislabeled images, used the same model as Trial 2, and tested multiple hyperparameters.

Results

- ◆ Train Dice: **0.8550**
- ◆ Validation Dice: **0.8641**
- ◆ Test Dice: **0.8690**

Observation: Notable improvement, especially on the test set.

Trial 4: Confidence Threshold and More Epochs

Justification: Introducing a confidence threshold and more epochs aimed to enhance prediction reliability and model convergence.

→ Added a confidence threshold of 0.7, increased epochs to 40, and used a learning rate of 0.0005.

Results

◆ Train Dice: **0.9214**

◆ Test Dice: **0.9240**

Observation: Significant improvement.

Further trying of tuning (lower learning rate or more epochs) of hyperparameters showed no enhancement, prompting the exploration of a more complex model (UNet).

Trial 5: UNet (with Early Stopping)

Justification: UNet was chosen for its strong performance in segmentation tasks, with early stopping to optimize training efficiency.

→ Switched to UNet, used a confidence threshold of 0.7, and applied early stopping with patience=7 to save time.

Results:

◆ Train Dice: **0.9296**

◆ Validation Dice: **0.9503**

◆ Test Dice: **0.9358**

Observation: Better results, so we proceeded with UNet.

Trial 6: UNet with Pretrained ResNet Encoder

Justification: A pre-trained ResNet encoder was used to leverage transfer learning for improved feature extraction.

→ Used UNet with a pre-trained ResNet encoder and the same 0.7 threshold.

Results

◆ Train Dice: **0.9300**

◆ Validation Dice: **0.9523**

◆ Test Dice: **0.9467**

Observation: Improved results as we're:

1. leveraging knowledge from models pre-trained on a large dataset
2. Adapting them to our specific task (segmentation)
3. Fine-tuning the weights on our data

Then, we continued with **pre-trained UNet**.

Trial 7: Lowering Confidence Threshold

Justification: A lower threshold was tested to evaluate the trade-off between recall and precision in predictions.

→ Lowered the confidence threshold to 0.3 using the same model.

Results

- ◆ Train Dice: **0.9230**
- ◆ Validation Dice: **0.9216**
- ◆ Test Dice: **0.9226**

Observation:

- **Lower Threshold → More Predictions Included → More Wrong Ones**
- **Lower threshold:** Includes many predictions, even uncertain ones. This increases recall but also allows more incorrect predictions into your metrics.
- According to the dice equation, as FP and FN go up due to wrong predictions being included, Dice goes down.

Trial 8: DeepLabV3 with Different Backbones

Justification: DeepLabV3 was explored for its advanced segmentation capabilities, despite memory challenges.

- Tested DeepLabV3 with a ResNet pre-trained encoder and got results similar to UNet
- Testing with EfficientNet-B7 backbone. Faced memory issues, so we reduced batch size and image size to 70% of the original.

Results (EfficientNet-B7)

- ◆ Train Dice: **0.8407**
- ◆ Validation Dice: **0.8212**

Observation: Significantly worse results compared to UNet.

Trial 9: UNet with ResNet and Further Data Filtering

Justification: Further data filtering was tested to minimize noise and maximize model performance with

UNet-ResNet.

- Returned to UNet with ResNet, and threshold of 0.7, and filtered additional mislabeled data misclassified during training.

Results

- ◆ Train Dice: **0.9516**
- ◆ Validation Dice: **0.9660**
- ◆ Test Dice: **0.9660**

Observation: Filtering noisy data significantly improved performance.

Trial 10: Modified Dice Calculation

Justification: The modified Dice calculation by the TA which includes only foreground

- Used UNet with ResNet, threshold of 0.7, but modified Dice calculation to only consider foreground pixels (ignoring true negatives).

Results

- ◆ Train Dice: **0.8400**
- ◆ Validation Dice: **0.8639**
- ◆ Test Dice: **0.8519**

Observation: Scores dropped significantly due to excluding true negatives, focusing only on correctly classified foreground pixels.

Trial 11: UNet with DenseNet121 Backbone

Justification: DenseNet121 was chosen for its dense connectivity, potentially improving feature reuse and segmentation.

- Used UNet with DenseNet121 backbone and a learning rate scheduler (which have lowered the learning rate from 0.0005 to 0.000125 when performance plateaued).

Results

- ◆ Train Dice: **0.8778**
- ◆ Validation Dice: **0.8766**

Observation: Moderate performance, needs further exploration.

Trial 12: Switching to Dice Loss

Justification: Dice Loss was adopted to align training loss with the actual evaluation metric, for better learning.

- We noticed evaluation was based on the Dice coefficient, so switched from CrossEntropyLoss to Dice Loss. Tested with ResNet34 and DenseNet121 backbones, starting with a low learning rate for better learning.

Results (Resnet34, low initial learning rate)

- ◆ Train: **0.8689**
- ◆ Validation: **0.8752**

Results (DenseNet121, low initial learning rate)

- ◆ Train Dice: **0.9059**
- ◆ Validation Dice: **0.9071**
- ◆ Test Dice: **0.8915**

Observation: Significant improvement with Dice Loss, especially with DenseNet121. Proceeded to enhance **UNet with DenseNet121**.

Trial 13: Image Size and Channel Adjustments

Justification: Smaller image sizes and channels 1 and 3 were tested to address size inconsistencies between train and test and consider bands and mask correlation.

- Noticed inconsistent test image sizes, so trained on smaller 256x256 images. Found channels 1, 2, and 3 were more correlated with masks, then using channels 1 and 3, and two thresholds:
- ◆ **Threshold 0.7:**
 - Train: Acc: **0.9541**, Dice: **0.9274**
 - Validation: Acc: **0.9462**, Dice: **0.9203**
 - Test: Acc: **0.9431**, Dice: **0.9116**
 - ◆ **Threshold 0.85:**
 - Train: Acc: **0.9464**, Dice: **0.9138**
 - Validation: Acc: **0.9438**, Dice: **0.9126**
 - Test: Acc: **0.9408**, Dice: **0.9059**

Observation: Channels 1 and 3 performed better on the test set.

Final Model Selection

We evaluated the last three trials, this is what we got on the leaderboard:

- UNet with DenseNet, 256x256, channels 1, 3: **0.7847**
- UNet with DenseNet, 512x512, channels 1, 4: **0.7900**
- UNet with DenseNet, 256x256, channels 1, 3, threshold 0.85: **0.7740**

Justification: The final model was selected based on leaderboard performance

Based on these results, we selected UNet with a DenseNet backbone, channels 1 and 4, and an image size of 512x512 as the final model due to its superior leaderboard performance.

VII. Enhancements and Future Work

The suggestions below aim to enhance model performance, robustness, and computational efficiency while tackling challenges such as mislabeled data and hardware constraints.

1. Improved Mislabeled Data Filtering

- Develop a semi-automated pipeline using clustering or anomaly detection to identify and flag potential mislabeled images for review. This could involve training a lightweight model to detect inconsistencies between images and masks.
- Expected Benefit: Cleaner data will enhance model generalization and reduce confusion during training.

2. Advanced Data Augmentation

- Apply targeted augmentations such as random brightness/contrast adjustments, rotations, and Gaussian noise to simulate real-world variability.
- Expected Benefit: Enhanced model robustness, particularly for edge cases like thin clouds or bright surfaces, which classical methods like Fmask struggled with.

3. Model Ensemble

- Create an ensemble of UNet (DenseNet121), UNet (ResNet), and DeepLabV3+ (ResNet) using majority voting or weighted averaging of predictions.
- Expected Benefit: Increased robustness and potentially higher Dice scores by leveraging complementary strengths of different architectures.

4. Multi-Resolution Training

- Implement a multi-resolution training strategy where the model is exposed to images at different scales (e.g., 256x256, 512x512) during training.
- Impact: Improved adaptability to varying image sizes, potentially closing the gap between internal test scores (Dice: 0.9116) and leaderboard scores (0.7900).

5. Model Exploration

- Experiment with different models and efficient backbones like
 - MobileNetV3 / EfficientNet-Lite such / GANs
 - Attention Mechanisms: Add Attention Gates or SE blocks (Squeeze-and-Excitation) into the decoder or skip connections to allow the model to focus on cloud regions.

6. Incorporate Additional Bands

- Test models with combinations of channels 1, 2, 3, and 4, or incorporate spectral indices as additional input features, this may enhance feature representation.

VIII. Work Distribution

ID	Name	Workload
9211230	Mennatallah Ahmed	• Focused on model evaluation and Hyperparameter tuning • Implementation of DeepLabV3+ models • Contributed to data preprocessing pipelines, and documented results.
9211194	Moustafa Mohammed	• Handled preprocessing (misabeled data filtering, band selection) • Handled model training for UNet-based models.
9211206	Mostafa Hani	• Conducted EDA (image visualization, mask distribution) • Implemented Random Forest,
9213024	Mohammed Alomar	• Led literature review • Contributed to EDA (band correlation analysis).